

Class Namespace

Document: P0223R0
Date: 2016-02-09
Project: ISO/IEC JTC1 SC22 WG21 Programming Language C++
Audience: Evolution Working Group
Author: Matthew Woehlke (mwoehlke.floss@gmail.com)

Abstract

This proposal provides a new language feature, "class namespace", as a shortcut for providing a series of definitions belonging to a class scope, similar to the manner in which a traditional namespace can provide a series of definitions belonging to a namespace scope.

This proposal is effectively a continuation / resurrection of [N1420](#) which was tagged for further consideration without being either accepted or rejected. However, much of the text of this proposal was written prior to the author's discovery of the same.

In contrast to N1420, we avoid use of the term "reopen", which implies the ability to add to a class. Class definitions are currently closed; members may only be added to a class during the initial definition thereof (template specialization notwithstanding). Although many have asked for the ability to add members to classes after the definition, such proposals are generally not well received. Although it is not the intent of this proposal to categorically forbid any such future direction, we also recognize these concerns and specifically do not wish to suggest any movement in that direction.

Contents

- [Abstract](#)
- [Rationale](#)
- [Proposal](#)
- [Specification](#)
- [Interactions](#)
- [Additional Examples](#)
- [Discussion](#)
 - [Syntax](#)
 - [Inline](#)
- [Acknowledgments](#)
- [References](#)

Rationale

[Don't Repeat Yourself](#) (DRY) is a well known principle of software design. However, there are certain instances when providing definitions of class members that can fall prey to repetition, to the detriment of readability and maintainability.

We will present, as a particularly egregious example, a complicated template class:

```
template <typename CharType, typename Traits, typename Allocator>
class MyString { ... };
```

There are strong reasons why method definitions should not be inline. For starters, they inhibit readability; it is difficult to quickly parse the interface — especially the public interface — as declarations and definitions are necessarily interleaved. Additionally, they are *inline*, which results in all manner of compile time and cross-version compatibility issues. Even for template classes, it is sometimes preferred to keep definitions in a separate TU (e.g. extern templates with only specific, exported explicit instantiations).

The problem that arises is the necessity to repeat a long prefix for all definitions provided outside of the class definition. For example:

```
template <typename CharType, typename Traits, typename Allocator>
MyString<CharType, Traits, Allocator>::MyString(...)
{ ... }
```

This repetition increases the space over which accidental errors may be introduced, and increases the work required for refactoring. The problem is compounded for templates within templates.

This is a real, extant problem. Presumably as a result of this overhead, some authors will use only inline definitions of methods, which can make it difficult to separate implementation details — which are often unnecessary noise for a user trying to understand a class — from a class's interface. Other authors may resort to separating return types, template parameters, class names, and method names, placing each on separate lines, resulting in method headers that are four lines long even before the argument list is considered. In the latter case, the need to repeat the class prefix is frustrating and, in the author's opinion, unnecessary.

(See https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/e0_ceXFQX-A for additional discussion.)

It is also worth noting that this situation is inconsistent with namespaces. Given a function declared in a namespace:

```
namespace Foo
{
    void foo();
}
```

...there are currently two ways to provide the definition:

```
// Method 1: fully qualified
void Foo::foo() { ... }

// Method 2: namespace scope
namespace Foo
{
    void foo() { ... }
}
```

There is currently no equivalent to the second form for class members. This proposal would remove this inconsistency.

Proposal

This proposal is to eliminate the redundancy by introducing a new "class scope" syntax, as follows:

```
template <...> // optional; only used for template classes
namespace class Name
{
    // definitions of class members
}
```

The effect of this scope is to treat each member definition (variable or method) as if it were prefixed by the class template specification and name. Specifically, these two codes would be exactly equivalent:

```
// Declarations
class Part { enum Type { ... }; ... };

template <typename T> class List { ... };

// Existing syntax
Part::Part(...) { ... }
Part::Type Part::type(...) { ... }
int Part::radius = ...;

template <typename T> List<T>::List(...) { struct iterator { ... }; ... }
template <typename T> List<T>& List<T>::operator=(List<T> const& other) { ... }
template <typename T> typename List<T>::iterator List<T>::find(...) { ... }

// Proposed syntax
namespace class Part {
    Part(...) { ... }
    Type type() { ... }
    int radius = ...;
}

template <typename T>
namespace class List {
    List(...) { ... }
    List& operator=(List const& other) { ... }
    iterator find(...) { ... }
}
```

Following the introduction of the scope (i.e. the keywords `namespace class`), the template parameters shall be implicitly applied to

the class name and any subsequent mention of the class name that does not have an explicit argument list. It shall be an error to provide an argument list for the introducing class name except in the case of specialization. Type name look-up within the scope shall additionally consider the class scope first (note in the above example the use of `Type` without the `Part::` qualifier). (These rules should be applied in the same manner as for a class definition. Note that this only affects non-trailing return types, as other types already use the class scope for type resolution.)

Some consequences of the scope acting simply as a name transformation should be noted. First, such a scope can be "opened" on the same class name any number of times. Second, definitions in a class name scope may be mixed with traditional, fully qualified definitions (provided that no definitions are duplicated, as always). Third, an empty scope is permissible as long as the named class is recognized. Last, but perhaps most important, the scope does not permit the addition of members not present in the class definition, nor in general does it allow the user to accomplish anything that could not be accomplished otherwise.

Additionally:

- While `namespace class` is being used for illustrative purposes, `namespace struct` and `namespace union` shall also be permitted, and shall provide equivalent function. (In general, the use of "class" throughout should be understood to mean any class-type.)
- Use of a class name scope requires that the named class has been defined. Forward declaration is not sufficient.
- Nested class name scopes are permitted.
- Any members that may legally be defined using their qualified name may be defined within a class name scope. This includes member types, member functions, and static member variables.
- As with traditional namespaces, a `;` is not required following the closing `}`.
- Access modifiers are not allowed in a class name scope. The `virtual` and `static` modifiers are not allowed in a class name scope. (None of these are allowed outside of a class definition, and the class name scope is not a class definition.)
- A class name scope may not add class members to a class definition.
- This proposal does not affect `using` directives. (A `using` directive on a class name scope remains illegal.)

Specification

The most straight forward way in which to describe this feature is with a syntax transformation. Specifically, the syntax:

```
[<template_specification>] namespace class <name> {  
    [<type>] <member_name><...>  
}
```

...shall be equivalent to:

```
[<template_specification>] [<type>] <name>::<member_name><...>
```

...for each `<member_name>` in the scope. Rules for interpretation of members within a class name scope, and for what sorts of code is permitted or ill-formed, may all be derived directly from this transformation. Type resolution for the return type (where applicable) shall proceed according to the same rules that would apply within the class definition.

Interactions

The token sequences `namespace class`, `namespace struct` and `namespace union` are currently ill-formed, so no existing code would be affected by this proposal. This proposal does not make any changes to other existing language or library features (although implementations would be free to make use of it in their standard library implementations, should they desire to do so).

Additional Examples

This feature is particularly useful for template members of template classes, including nested template types:

```
template <typename T> class Foo  
{  
    template <typename U> void foo(U);  
    template <typename U> class Bar { Bar() };  
};  
  
template <typename T> namespace class Foo  
{  
    template <typename U> void foo(U) { ... }  
  
    template <typename U> class Bar  
    {
```

```

    Bar() { ... }
}

// Compare to the old syntax:
template <typename T>
template <typename U>
void Foo<T>::foo<U>(U) { ... }

template <typename T>
template <typename U>
void Foo<T>::Bar<U>::Bar() { ... }

```

Per the transformation rule, it works with specializations, as one would expect:

```

template <> namespace class Foo<int>
{
    ...
}

```

(Note that this is allowed with or without a specialization of `Foo<int>`, just as it is currently permitted to specialize class members without specializing the entire class definition. Naturally, if the class definition is specialized, then definitions in the corresponding class name scope must match members declared in said specialization.)

Discussion

Syntax

The proposed syntax for introducing the scope is open for debate. Alternative suggestions include:

1. `class namespace <name>`
2. `namespace <classname>`
3. Introduction of a new contextual keyword, e.g. `class <name> implementation`.
4. Introduction of a new (global) keyword, e.g. `implement class <name>`.

The author considers #1 to be very nearly as good as the suggested syntax. #2 is okay, but risks confusion, as the reader must know a priori if the named scope is a class (the #2 syntax would only introduce a class name scope if the identifier following the `namespace` keyword is an already declared class-type). #3 is of similar quality to #2; it lacks the ambiguity problem, but the indication that "something is different" occurs later, and it does require a new (albeit contextual) keyword. #4 has the advantage of maximum possible clarity, but introducing new keywords without breaking existing code is always tricky.

We additionally feel that the proposed syntax is the most consistent with the current state of the language. It maintains the traditional order of tokens, e.g. compared to use of traditional namespaces. It uses tokens in an order than makes sense according to English grammar rules, i.e. *<verb> <adjective> <noun>* (with `namespace` here acting as a verb, indicating that a scope block is starting) with `namespace class Foo` comparable to e.g. "open blue ball".

Inline

Should `inline namespace class <name>` be permitted? The "inline namespace" concept does not make sense in this context. If it is permitted, it should be equivalent to including `inline` as part of every contained definition. The author's inclination is to forbid use of `inline` with `namespace class`.

Acknowledgments

This proposal is a continuation of [N1420](#) by Carl Daniel. It was originally written prior to the author's discovery of N1420. The original feature request that spawned this new proposal comes from John Yates. Miro Knejp and Péter Radics contributed valuable suggestions. Other contemporary participants include Larry Evans, Russell Greene, Bjorn Reese, Evan Teran and Andrew Tomazos. (The author also acknowledges prior discussion of a very similar feature: see https://groups.google.com/a/isocpp.org/d/msg/std-proposals/xukd1mgd21I/uHjx6YR_EnQJ and <https://groups.google.com/a/isocpp.org/d/msg/std-proposals/xukd1mgd21I/gh5W0KS856oJ>.)

References

- [N1420](#) Class Namespaces

